- Upload to Google Drive or YouTube and share the link.

02

PROBLEM STATEMENT 2

P2P Web Share — Direct Browser-to-Browser File Transfer

WebRTC | Node.js | React.js

§ 1 OBJECTIVE

Traditional file-sharing services rely on central servers, resulting in heavy bandwidth costs and storage limitations. The objective of this project is to build a lightweight, decentralized P2P file sharing web application. Using this platform, users can drop a file to generate a unique share room link. Recipients who open this link connect directly to the sender's browser to stream the file. A lightweight central signaling server coordinates the initial handshake but never reads, processes, or stores any part of the file data.

§ 2 KEY FEATURES — CORE MVP

For a successful basic submission, the application must achieve stable, secure 1-to-1 direct transfers:

- **Share Room Creation:** Drag-and-drop zone to upload files (limit <50MB for standard browser memory) and generate a unique Room ID or invite link.
- **Signaling Handshake:** A simple Node.js / Socket.io signaling backend to coordinate initial WebRTC connection offers and answers.
- **Direct P2P Transfer:** Once connected, read the file using the browser FileReader API and transfer it directly to the receiver over WebRTC data channels.
- **Basic Chunk Verification:** Generate and verify a cryptographic hash (e.g., SHA-256) of the file chunks before and after transfer to guarantee zero data corruption.
- **Progress Indicators & Connection Status:** A real-time UI showing transfer percentage, transfer speed (MB/s), and active connection status.
- **Graceful Disconnect Handling:** Ensure the application does not crash or freeze if a user closes a tab or disconnects. The UI should gracefully notify the remaining user of the connection drop.
- **Auto-Download:** Reassemble the incoming verified chunks in the receiver memory and automatically trigger a local file download when completed.

§ 3 BROWNIE POINTS / ADVANCED FEATURE EXTENSIONS (OPTIONAL)

Implement any of these advanced extensions to make your project stand out:

- **Multi-Peer Support (Mesh Swarming):** Allow a third peer joining the room to download different parts of the file from both the sender and the second peer simultaneously.
- **Large File Support (>500MB):** Bypass standard browser RAM limitations by writing incoming chunks directly to local storage using the Streams API paired with Origin Private File System (OPFS) or IndexedDB.
- **Zero-Knowledge Encryption:** Encrypt the file chunks in the browser using the Web Crypto API (AES-GCM) before transmitting. Pass the decryption key purely via the URL hash (e.g., /#key=...) so the signaling server never has access to the raw file data.
- **Connection Churn Recovery (Auto-Resume):** Implement a state-tracking handshake so that if a connection drops mid-transfer, the download can automatically pause and resume from the last verified chunk once the peer reconnects, rather than restarting from 0%.

§ 4 TECH STACK SUGGESTIONS

LAYER	SUGGESTED TECHNOLOGIES
Frontend	React.js, Tailwind CSS
P2P Communication	WebRTC API, or wrapper libraries like PeerJS or Simple-Peer
Backend Signaling	Node.js + Express.js + Socket.io
Hosting	Vercel / Netlify (Frontend), Render / Railway (Backend)

§ 5 SUBMISSION INSTRUCTIONS

1. GITHUB REPOSITORY

- Public repo containing clean, commented frontend and backend code.
- Include a `README.md` with project description, setup instructions, and deployment links.

2. DEMO VIDEO (~3 MINUTES)

- A short screen recording demonstrating a live file transfer between two different browser windows or devices.

- Upload to Google Drive or YouTube and share the link.

— NOTICE —

Submissions found to be copied from GitHub or other teams will be disqualified. All work must be original and done by you / your team. Feel free to use open-source tools and libraries, but ensure your implementation and logic are entirely your own. Submissions utilising copy-pasted baseline WebRTC templates without customised UI, active progress indicators, and proper error feedback will be disqualified.